



2º Ciclo - Fundamentos do Processamento de Dados

Python para Processamento de Dados [26E2_4]

Conjuntos (Sets)

Conjuntos são coleções não-ordenadas, mutáveis e sem itens duplicados.

Criação de Conjuntos

```
# com {}

frutas = {'maca', 'banana', 'caju', 'manga', 'jabuticaba', 'jabuticaba'}
print(frutas)
```

```
{'maca', 'banana', 'jabuticaba', 'manga', 'caju'}
```

```
# elementos repetidos sao automaticamente removidos
letras = {'a', 'b', 'c', 'a', 'b', 'd', 'd', 'a'}
print(letras)
```

```
{'d', 'c', 'a', 'b'}
```

```
# conjunto vazio, cuidado!. Nao se pode usar {} para criar um conjunto vazio.
```

```
conjunto_vazio = set()print(type(conjunto_vazio))
```

```
<class 'set'>
```

```
# criar conjuntos a partir de outras colecoes
```

```
vogais = set('aeiou')
print(vogais)

valores_unicos = set([1,2,4,6,2,4,6,7, 2, 4, 2, 1])
print(valores_unicos)
```

```
{'i', 'e', 'u', 'o', 'a'}
{1, 2, 4, 6, 7}
```

Operações com Conjuntos

```

frutas = {'maca', 'banana', 'caju', 'manga', 'jabuticaba'}

# adicionando elementos
frutas.add('uva')
print(frutas)
print(80*'-')

# adicionando multiplos elementos
frutas.update(['pera', 'abacaxi'])
print(frutas)
print(80*'-')

# removendo elementos
frutas.remove('banana') # gera erro se o elemento nao existir
frutas.discard('manga') # nao gera erro se o elemento nao existir
elemento = frutas.pop()
print(f'Fruta retirada do conjunto de frutas:', elemento)

print(frutas)
print(80*'-')

```

```

{'maca', 'uva', 'banana', 'jabuticaba', 'manga', 'caju'}

```

```

{'maca', 'uva', 'banana', 'abacaxi', 'jabuticaba', 'pera', 'manga', 'caju'}

```

```

Fruta retirada do conjunto de frutas: maca
{'uva', 'abacaxi', 'jabuticaba', 'pera', 'caju'}

```

```

# removendo um elemento que nao existe com remove
print(frutas)
print(80*'-')
frutas.remove('ata')
print(frutas)
print(80*'-')

```

```

{'uva', 'abacaxi', 'jabuticaba', 'pera', 'caju'}

```

❗ Execution error

KeyError: 'ata'

```

-----
KeyError                                Traceback (most recent call last)
Cell In[86], line 4
      2 print(frutas)
      3 print(80*'-')
----> 4 frutas.remove('ata')
      5 print(frutas)
      6 print(80*'-')

```

KeyError: 'ata'

Hide error details

```

# removendo um elemento que nao existe com discard
print(frutas)
print(80*'-')
frutas.discard('ata')
print(frutas)
print(80*'-')

```

```

{'uva', 'abacaxi', 'jabuticaba', 'pera', 'caju'}

```

```

{'uva', 'abacaxi', 'jabuticaba', 'pera', 'caju'}

```

```
# limpando um conjunto
print('Tamanho do conjunto de frutas:', len(frutas))
print(frutas)
print(80*'-')
frutas.clear()
print('Tamanho do conjunto de frutas:', len(frutas))
print(frutas)
print(80*'-')
```

Tamanho do conjunto de frutas: 5
{'uva', 'abacaxi', 'jabuticaba', 'pera', 'caju'}

Tamanho do conjunto de frutas: 0
set()

Operações Matemáticas

```
A = {1, 2, 3, 4, 5}
B = {4, 5, 6, 7, 8}
```

```
# uniao
uniao = A | B
print(uniao) # {1,2,3,4,5,6,7,8}
```

```
# intersecao
intersecao = A & B # {4, 5}
print(intersecao)
```

```
# diferenca (elemntos que pertence somente a um determinado conjunto)
diferenca_A_B = A - B # {1, 2, 3}
print(diferenca_A_B)
diferenca_B_A = B - A # {6, 7, 8}
print(diferenca_B_A)
```

{1, 2, 3, 4, 5, 6, 7, 8}
{4, 5}
{1, 2, 3}
{8, 6, 7}

Operações de Comparação

```
A = { i for i in range(10)}
B = { i for i in range(0, 21, 3)}
C = { i for i in range(2, 10, 2)}

print(f'Conjunto A: {A}')
print(f'Conjunto B: {B}')
print(f'Conjunto C: {C}')
```

Conjunto A: {0, 1, 2, 3, 4, 5, 6, 7, 8, 9}
Conjunto B: {0, 3, 6, 9, 12, 15, 18}
Conjunto C: {8, 2, 4, 6}

```
# 1o: verificar se um conjunto contem um outro conjunto.
# Ex.: A contem B, verificar se todos os elementos do conjunto B tambem sao elementos do conjunto A
A_contem_C = A > C
print(f'A contem C: {A_contem_C}, ou seja, {"sim" if A_contem_C else "nao"}') # True
C_contem_A = C > A
print(f'C contem A: {C_contem_A}, ou seja, {"sim" if C_contem_A else "nao"}') # False
```

A contem C: True, ou seja, sim
C contem A: False, ou seja, nao

2o: igualdade entre conjuntos

```
A_intersecao_C = A & C
```

o conjunto A intersecai C eh igual ao conjunto C?

```
resultado = A_intersecao_C == C
```

```
print(f'o conjunto A intersecao C eh igual ao conjunto C? {resultado}')
```

```
print(f'0 conjunto B eh igual ao conjunto A? {B == A}')
```

o conjunto A intersecao C eh igual ao conjunto C? True

0 conjunto B eh igual ao conjunto A? False

verificando se um conjunto eh subconjunto de outro:

```
C_eh_subconjunto_de_A = C.issubset(A)
```

```
print(f'0 conjunto C eh um subconjunto de A?: {C_eh_subconjunto_de_A}, ou seja, {"sim" if C_eh_subconjunto_de_A else "nao"}')
```

```
A_contem_C = A > C
```

```
print(f'A contem C: {A_contem_C}, ou seja, {"sim" if A_contem_C else "nao"}') # True
```

0 conjunto C eh um subconjunto de A?: True, ou seja, sim

A contem C: True, ou seja, sim

verificando se um conjunto eh superconjunto de outro:

```
A_eh_super_conjunto_C = A.issuperset(C)
```

```
print(f'0 conjunto A eh um super conjunto de C?: {A_eh_super_conjunto_C}, ou seja, {"sim" if A_eh_super_conjunto_C else "nao"}')
```

0 conjunto A eh um super conjunto de C?: True, ou seja, sim

```
A = { i for i in range(2, 21, 2)}
```

```
B = { i for i in range(3, 21, 2)}
```

```
print(f'Conjunto A: {A}')
```

```
print(f'Conjunto B: {B}')
```

Conjunto A: {2, 4, 6, 8, 10, 12, 14, 16, 18, 20}

Conjunto B: {3, 5, 7, 9, 11, 13, 15, 17, 19}

```
A = { i for i in range(2, 21, 2)}
```

```
B = { i for i in range(3, 21, 2) if (i % 2) != 2}
```

```
print(f'Conjunto A: {A}')
```

```
print(f'Conjunto B: {B}')
```